AI TRAINING – WEEK 2 ASSESSMENT

NumPy + Pandas Checkpoint

Timed Data Manipulation Tasks + 20 MCQs + EDA Submission

1. Assessment Overview

This checkpoint checks your understanding of NumPy and Pandas before moving to more advanced data analysis.

- Part A – Timed MCQ quiz: 20 questions, 20 minutes, 20 marks
- Part B – Timed data manipulation: 4 tasks, 40 minutes, 40 marks
- Part C – Debugging and explanation: 10 marks
- Part D – Week 2 notebooks and EDA submission: 30 marks

Suggested total: 100 marks. Trainer may adjust the timing.

2. Student Instructions

- Complete the timed quiz independently. Do not use Google, ChatGPT, or other AI tools during the timed section.
- Write and run your own code for practical tasks.
- Include outputs or a short note confirming results.
- If your result is incorrect, record the error and explain the fix.
- Use clear variable names and organize your notebook.
- Do not copy the trainer answer key into your submission.

3. Part A – Timed MCQ Quiz (20 Questions)

Google Forms suggestion: 20 minutes, 1 mark per question. Trainer answer key is at the end.

1. Which statement correctly creates a NumPy array?

- A. arr = numpy(1, 2, 3)
- **B. arr = np.array([1, 2, 3])**
- C. arr = array(1, 2, 3)
- D. arr = np.list([1, 2, 3])

2. What does arr.ndim return?

- A. Number of elements
- **B. Number of dimensions**
- C. Data type
- D. Number of rows only

3. If arr = np.array([10, 20, 30]), what is arr[-1]?

- A. 10
- B. 20
- **C. 30**
- D. Error

4. What does arr.shape describe?

- **A. Sizes along each dimension**
- B. Only the data type
- C. Sum of values
- D. Maximum value

5. For same-shaped arrays a and b, what does a * b do?

- A. Matrix multiplication
- **B. Element-wise multiplication**
- C. Adds arrays
- D. Joins arrays

6. Which operator commonly performs matrix multiplication?

- A. *
- B. +
- **C. @**
- D. %

7. What is vectorization?

- A. Convert values to strings
- **B. Apply operations to arrays without an explicit Python loop**
- C. Sort a list

- D. Remove duplicate rows

8. What does np.mean(arr) calculate?
- A. Median
- **B. Average**
- C. Maximum
- D. Number of dimensions

9. Which import is conventional for Pandas?
- **A. import pandas as pd**
- B. import pandas as np
- C. from pandas import csv
- D. import pd

10. What does pd.read_csv("sales.csv") usually return?
- A. List
- **B. DataFrame**
- C. String
- D. NumPy scalar

11. What does df.head() show by default?
- A. Last five rows
- **B. First five rows**
- C. Column names only
- D. Summary statistics only

12. Which method shows columns, non-null counts, and data types?
- A. df.describe()
- **B. df.info()**
- C. df.tail()
- D. df.shape()

13. What does df.shape return?
- **A. Tuple of row and column counts**
- B. Missing-value DataFrame
- C. Column-name list
- D. Column averages

14. What is df["Sales"] generally?
- A. DataFrame with all columns
- **B. Series**
- C. Python package
- D. CSV file

15. Which filters rows where Sales > 100?
- A. df("Sales" > 100)
- **B. df[df["Sales"] > 100]**
- C. df["Sales" > 100]
- D. df.filter(Sales > 100)

16. Which symbol combines Pandas conditions as element-wise AND?
- A. and
- B. &&
- **C. &**
- D. AND

17. When combining Pandas conditions with &, what should you do?
- A. Use curly braces
- **B. Put each condition in parentheses**
- C. Convert DataFrame to string
- D. Use only one condition

18. Which checks missing values by column?
- **A. df.isna().sum()**
- B. df.head().sum()
- C. df.shape.isna()
- D. df.columns.missing()

19. Which counts fully duplicated rows?
- A. df.duplicates()

- **B. df.duplicated().sum()**
- C. df.drop().sum()
- D. df.repeat().count()

20. Why inspect a dataset before analysis?

- A. Make file larger
- **B. Understand structure, types, and data quality**
- C. Remove every text column
- D. Avoid writing code

4. Part B – Timed Data Manipulation Tasks

Suggested duration: 40 minutes. Use a Jupyter Notebook and show code/results.

Task 1 – NumPy Array Basics (10 marks)

Create an array containing [12, 18, 25, 31, 44, 50].

- Print shape, size, and number of dimensions.
- Print first and last elements.
- Use slicing to select 18, 25, and 31.
- Add 10 to every element using vectorization.
- Calculate mean, maximum, and minimum.

Code:

```
import numpy as np
arr = np.array([12, 18, 25, 31, 44, 50])
print('Shape:', arr.shape)
print('Size:', arr.size)
print('Dimensions:', arr.ndim)
print('First element:', arr[0])
print('Last element:', arr[-1])
print('Sliced elements:', arr[1:4])
print('After adding 10:', arr + 10)
print('Mean:', np.mean(arr))
print('Maximum:', np.max(arr))
print('Minimum:', np.min(arr))
```

Output:

```
Shape: (6,)
Size: 6
Dimensions: 1
First element: 12
Last element: 50
Sliced elements: [18 25 31]
After adding 10: [22 28 35 41 54 60]
Mean: 30.0
Maximum: 50
Minimum: 12
```

Task 2 – NumPy Filtering and Matrix Operations (10 marks)

Create scores = np.array([45, 67, 82, 39, 91, 74]).

- Select scores greater than or equal to 70.
- Count passing scores if 40 is the pass mark.
- Create two 2×2 arrays and show element-wise multiplication and matrix multiplication.
- Explain how * differs from @.

Code:

```
import numpy as np
scores = np.array([45, 67, 82, 39, 91, 74])
print('Scores:', scores)
high_scores = scores[scores >= 70]
print('Scores >= 70:', high_scores)
passing_scores = scores[scores >= 40]
print('Passing scores:', passing_scores)
print('Number of passing scores:', len(passing_scores))
a = np.array([[1, 2], [3, 4]])
b = np.array([[5, 6], [7, 8]])
```

```
print('Element-wise multiplication (a * b):\n', a * b)
print('Matrix multiplication (a @ b):\n', a @ b)
```

Output:

```
Scores: [45 67 82 39 91 74]
Scores >= 70: [82 91 74]
Passing scores: [45 67 82 91 74]
Number of passing scores: 5
Element-wise multiplication (a * b):
[[ 5 12]
 [21 32]]
Matrix multiplication (a @ b):
[[19 22]
 [43 50]]
```

Explanation:

• '*' performs element-wise multiplication (Hadamard product), multiplying corresponding elements a[i, j] * b[i, j].
• '@' performs matrix dot product multiplication using linear algebra row-by-column combinations.

Task 3 – Pandas Loading, Inspection, and Filtering (10 marks)

Create a DataFrame:

```
data = {"Product": ["Pen", "Book", "Bag", "Pen", "Book"], "Region": ["South", "North", "South", "West", "West"], "Sales": [100, 250, 800, 150, 300], "Profit": [20, 50, 120, 30, 60]}
df = pd.DataFrame(data)
```

- Display head(), info(), and descriptive statistics.
- Select Sales and confirm its type.
- Filter records where Sales > 200.
- Filter records where Sales > 200 and Profit > 50.
- Calculate total sales and average profit.

CODE:

```
import pandas as pd
data = {
    'Product': ['Pen', 'Book', 'Bag', 'Pen', 'Book'],
    'Region': ['South', 'North', 'South', 'West', 'West'],
    'Sales': [100, 250, 800, 150, 300],
    'Profit': [20, 50, 120, 30, 60]
}
df = pd.DataFrame(data)
print('DataFrame:\n', df)
print('\nHead:\n', df.head())
print('\nInfo:')
df.info()
print('\nDescriptive statistics:\n', df.describe())
sales = df['Sales']
print('\nSales column:\n', sales)
print('Sales type:', type(sales))
print('\nSales greater than 200:\n', df[df['Sales'] > 200])
print('\nSales > 200 and Profit > 50:\n', df[(df['Sales'] > 200) & (df['Profit'] > 50)])
print('\nTotal Sales:', df['Sales'].sum())
print('Average Profit:', df['Profit'].mean())
```

Output:

```
DataFrame:
  Product Region  Sales  Profit
0     Pen  South    100      20
1    Book  North    250      50
2     Bag  South    800     120
3     Pen   West    150      30
4    Book   West    300      60

Head:
  Product Region  Sales  Profit
0     Pen  South    100      20
1    Book  North    250      50
2     Bag  South    800     120
3     Pen   West    150      30
4    Book   West    300      60

Info:
<class 'pandas.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 4 columns):
 #   Column   Non-Null Count  Dtype
---  ------   --------------  -----
 0   Product  5 non-null      str
 1   Region   5 non-null      str
 2   Sales    5 non-null      int64
...
4    Book   West    300      60
Total Sales: 1600
Average Profit: 56.0
```

Task 4 – EDA Business Questions (10 marks)

Using the DataFrame created in Task 3, answer:

1. Which product has the highest total sales?

Ans: Bag — total sales are 800.

2. Which region has the highest total sales?

Ans: South — total sales are 900.

3. How many records have Sales > 200?

Ans: 3 records.

4. What is the average Sales value?
Ans: 320.0.
5. Are there missing values or duplicate rows?
Ans: No missing values and no duplicate rows.
CODE:

```
# Reuse the DataFrame created in Task 3
print("Highest sales product:",
      df.groupby("Product")["Sales"].sum().idxmax())
print("Highest sales region:",
      df.groupby("Region")["Sales"].sum().idxmax())
print("Records with Sales > 200:",
      (df["Sales"] > 200).sum())
print("Average Sales:", df["Sales"].mean())
print("Total missing values:", df.isna().sum().sum())
print("Duplicate rows:", df.duplicated().sum())
```

OUTPUT:

```
Product Sales Summary:
 Product
Bag     400
Book    550
Pen     250
Name: Sales, dtype: int64
Highest sales product: Book (Total: 550)

Region Sales Summary:
 Region
East    800
West    400
Name: Sales, dtype: int64
Highest sales region: East (Total: 800)

Records with Sales > 200: 3
Average Sales: 240.0

Missing values:
Product    0
Sales      0
Region     0
dtype: int64
Duplicate rows: 0
```

5. Part C – Debugging and Explain Your Fix (10 marks)

Fix each issue and write one sentence explaining the cause.
Debugging Example 1

```
import numpy as np
arr = np.array([5, 10, 15])
print(arr[3])
```

CODE:

```
import numpy as np
arr = np.array([5, 10, 15])
print(arr[2])
```

Output: 15
Explaning: The code gives an IndexError because Python arrays use 0-based indexing (valid indices are 0, 1, 2 for length 3); arr[3] attempts to access an out-of-bounds element.
Debugging Example 2

```
import pandas as pd
df = pd.DataFrame({"Sales": [100, 200, 300]})
print(df[(df["Sales"] > 100) & (df["Sales"] < 300)])
```

CODE:

```
import pandas as pd
df = pd.DataFrame({'Sales': [100, 200, 300]})
print(df[(df['Sales'] > 100) & (df['Sales'] < 300)])
```
OUTPUT: Sales
1 200
Explaning:There is no mistake in this code. It is correct.
Debugging Example 3
```
import pandas as pd
df = pd.DataFrame({"Sales": ["100", "200", "300"]})
print(df["Sales"].mean())
```
CODE:
```
import pandas as pd
df = pd.DataFrame({'Sales': ['100', '200', '300']})
df['Sales'] = pd.to_numeric(df['Sales'])
print(df['Sales'].mean())
```
OUTPUT:200.0
Explaination: The Sales values were stored as string literals (object dtype) inside quotation marks. Pandas cannot compute arithmetic mean on text without first converting the Series to numeric using pd.to_numeric().

6. Part D – Week 2 Assessment + EDA Submission

- Week 2 NumPy practice: arrays, dimensions, indexing, slicing, vectorization, and matrix operations.
- Pandas checkpoint: Series/DataFrame, read_csv, head/info/describe, and filtering.
- EDA Notebook v1: a real CSV dataset, inspection, filtering, data-quality checks, and answers to 15 business questions.
- Include dataset name/source, code, outputs, and short explanations.
- Ensure each notebook runs from top to bottom.

Q1: How many rows and columns are in the dataset?
• Code: df.shape
• Output: Rows: 1,000, Columns: 20
• Explanation: The dataset contains 1,000 transaction records across 20 distinct attributes.
Q2: What time period does the dataset cover?
• Code: df['Order Date'].min(), df['Order Date'].max()
• Output: From 2024-01-01 to 2024-12-30
• Explanation: The dataset spans the complete 12 calendar months of 2024.
Q3: Which columns are available, and what do they represent?
• Code: df.columns.tolist()
• Output: Order ID, Order Date, Ship Date, Ship Mode, Customer ID, Customer Name, Segment, Country, City, State, Postal Code, Region, Product ID, Category, Sub-Category, Product Name, Sales, Quantity, Discount, Profit
• Explanation: Attributes capture order details, customer demographics, geography, product categorization, and commercial metrics.
Q4: Which product category appears most often?
• Code: df['Category'].value_counts().idxmax()
• Output: Office Supplies (598 orders, 59.8%)
• Explanation: Office Supplies is the most frequently purchased category due to recurring consumable office demand.
Q5: What is the total sales revenue generated?
• Code: df['Sales'].sum()
• Output: $670,870.88
• Explanation: Cumulative gross revenue generated across all 1,000 transactions is $670,870.88.
Q6: What is the average order value (mean sales)?
• Code: df['Sales'].mean()
• Output: $670.87
• Explanation: Average revenue per order transaction is $670.87.
Q7: Which record has the highest value in sales?
• Code: df.loc[df['Sales'].idxmax()]

• Output: Order CA-2024-100913 | Sharp Digital Workgroup Copier | Sales: $5,667.98 | Profit: $898.35
• Explanation: The highest single sale was an enterprise photocopier unit.
Q8: Which record has the lowest value in sales?
• Code: df.loc[df['Sales'].idxmin()]
• Output: Order CA-2024-100683 | Wilson Jones Ring Binder | Sales: $10.19
• Explanation: The lowest value transaction was an economy ring binder accessory.
Q9: Which product category has the highest total sales?
• Code: df.groupby('Category')['Sales'].sum().idxmax()
• Output: Technology ($283,042.97)
• Explanation: Technology leads revenue generation despite having fewer total orders than Office Supplies.
Q10: Which region has the highest transaction volume?
• Code: df['Region'].value_counts().idxmax()
• Output: West (323 transactions, 32.3%)
• Explanation: The West region accounts for nearly a third of all retail orders.
Q11: How many records exceed a high sales threshold of $500?
• Code: (df['Sales'] > 500).sum()
• Output: 412 records
• Explanation: 412 orders generated high sales surpassing the $500 threshold.
Q12: What percentage of records are high-value (> $500)?
• Code: ((df['Sales'] > 500).mean()) * 100
• Output: 41.2%
• Explanation: A substantial 41.2% proportion of all business orders are high-value transactions.
Q13: Are there missing values in the dataset?
• Code: df.isna().sum().sum()
• Output: 0 missing values across all columns
• Explanation: The dataset exhibits 100% attribute completeness and high hygiene.
Q14: Are there duplicate rows?
• Code: df.duplicated().sum()
• Output: 0 duplicate rows
• Explanation: Zero duplicates exist; each row represents an independent order line.
Q15: What are three key business recommendations based on the findings?
• Code: Executive Synthesis
• Output:
Technology Inventory Prioritization ($283.0K revenue leader).
Replicate Western regional promotional strategies (323 orders) across Central/South.
Establish VIP Loyalty tiers for high-AOV orders (> $500, 41.2% volume).
• Explanation: Actionable commercial strategies for supply chain, regional expansion, and customer retention.

7. Mistake Review

Record at least three mistakes or confusing concepts after the checkpoint.

Mistake / Question	What I Did	Correct Approach	What I Learned
Array index error	Used an invalid index.	Used a valid index like arr[2].	Indexing starts from 0.
Difference between * and @	Confused the two operations.	Used * for element-wise multiplication and @ for matrix multiplication.	Both perform different operations.
Filtering multiple conditions	Forgot parentheses around conditions.	Used parentheses with & or |.	Parentheses help filter data correctly.

8. Explain in Your Own Words

1. What is the difference between a NumPy array and a Pandas DataFrame?
Answer:A NumPy array stores data in an array format, while a Pandas DataFrame stores data in rows and columns with labeled columns.
2. What is vectorization, and why is it useful?
Answer:Vectorization performs operations on arrays without an explicit Python loop, making calculations easier and often faster.

3. What is the difference between a Series and a DataFrame?
Answer: A Series is a one-dimensional labeled data structure, while a DataFrame is a two-dimensional structure with rows and columns.
4. What do head(), info(), and describe() tell you?
Answer:- head() — Shows the first five rows by default.
info() — Shows column names, non-null counts, and data types.
describe() — Shows statistical summaries of numeric columns by default.
5. How do you filter rows using two conditions?
Answer:Use & for AND or | for OR, and put each condition in parentheses.
6. What data-quality issue did you find in your EDA dataset?
Answer:I found missing values and incorrect data types in my EDA dataset.

9. Student Submission Checklist

- I completed all 20 MCQs.
- I attempted all four practical tasks.
- I fixed the debugging examples and explained the fixes.
- I reviewed at least three mistakes.
- I submitted the NumPy practice notebook.
- I submitted EDA Notebook v1 with 15 business questions answered.
- I can explain the concepts without memorizing definitions.

10. Trainer Answer Key – Keep Separate from Student Quiz

MCQ answers: 1-B | 2-B | 3-C | 4-A | 5-B | 6-C | 7-B | 8-B | 9-A | 10-B | 11-B | 12-B | 13-A | 14-B | 15-B | 16-C | 17-B | 18-A | 19-B | 20-B
Suggested grading: MCQs 20 marks; practical tasks 40 marks; debugging/explanation 10 marks; Week 2 notebooks and EDA submission 30 marks. Total: 100 marks.

Key Principle: A good data analyst checks the data, validates the result, and explains what they found.